

Lab 4: Subroutine & Macro

- 1. 介紹 PIC18 記憶體架構 (Memory Organization)
  - Program Memory（程式記憶體）
  - Data Memory（資料記憶體）
- 2. 相關 SFR 與控制機制
  - 2.1 Program Counter (PC)
  - 2.2 Stack 機制（STKPTR、TOS）
- 3. 模組化設計：Subroutine 與 Macro
  - 3.1 Subroutine
  - 3.2 Macro（巨集）
  - 3.3 Macro vs. Subroutine 綜合比較
- 4. 乘法運算 (Multiply)
  - 4.1 Unsigned Multiplication（無號數乘法）
  - 4.2 Signed Multiplication（有號數乘法）

1. 介紹 PIC18 記憶體架構 (Memory Organization)

PIC18 的儲存空間主要分為兩大類別：

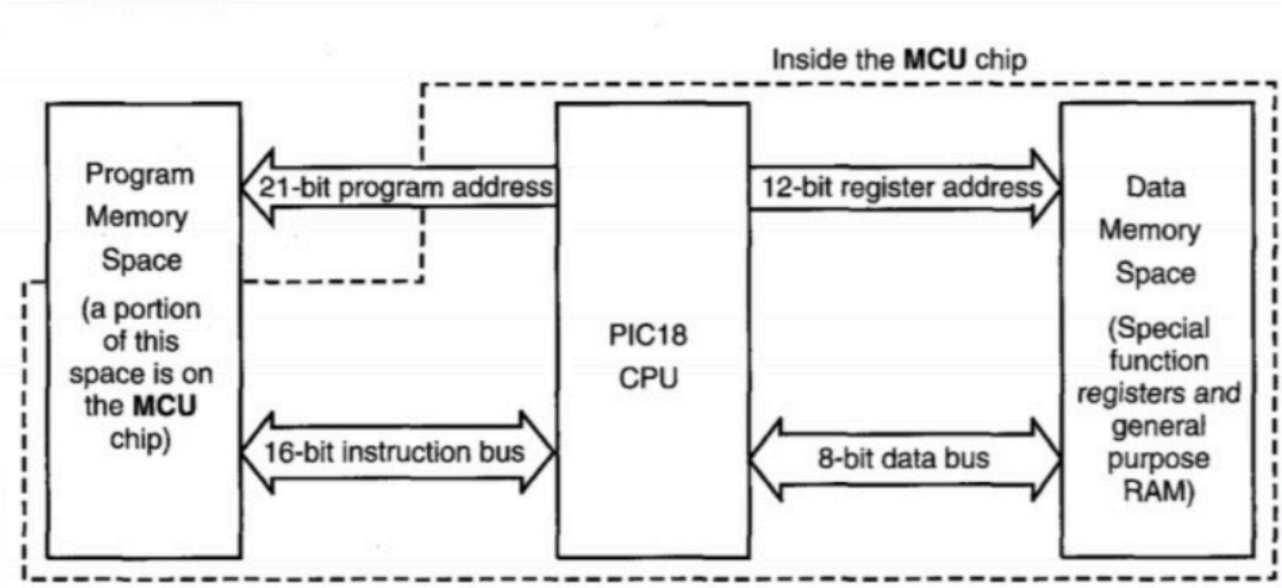


Figure 1.3 ■ The PIC18 memory spaces

Program Memory（程式記憶體）

- **用途**：存放撰寫的程式碼。
- **位址長度**：Program Address 為 **21-bit**（最多支援 2^{21} 個空間）。
- **單元容量**：每個儲存單元為 **8-bit**。
- **指令佔用**：PIC18 的每道指令大小為 **16-bit**，因此一個指令會**佔用 2 個連續的儲存單元**。

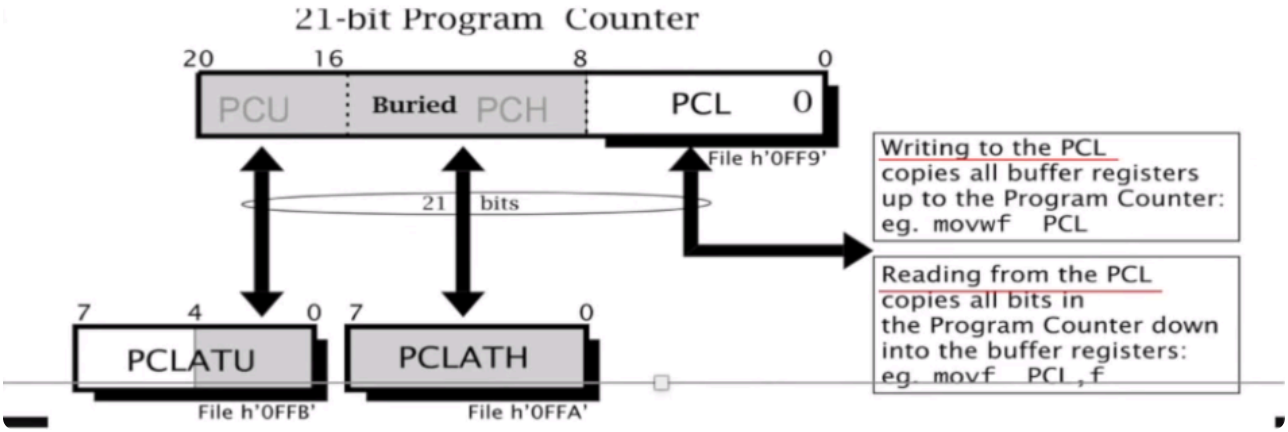
Data Memory（資料記憶體）

- **用途**：存放執行時的資料（例如 File Register、SFR 等）。
- **位址長度**：Data Address 為 **12-bit**（共 $2^{12} = 4096$ 個空間）。
- **單元容量**：每個儲存單元為 **8-bit**。

2. 相關 SFR 與控制機制

2.1 Program Counter (PC)

Program Counter (PC) 為 21-bit，用於存放**下一個要執行的指令位址**。



- **暫存器組成**：由三個暫存器組合而成（ `PCU` 、`PCH` 、`PCL` ）。
  - `PCL` （低位元）：可直接讀取與寫入。
  - `PCH` / `PCU` （高位元）：無法直接寫入，需透過 Latch 暫存器（ `PCLATH` / `PCLATU` ）輔助修改。
- 可透過直接修改 `PCL` 的值來寫 Loop（不一定要使用 `GOTO` ）。
- **注意**：標籤（Label）該行本身不算入指令長度。

> **為什麼 PC 裡面的值都是 2 的倍數？**
> 因為 PIC18 的指令大小為 **16-bit**，而記憶體的基本單位是 **8-bit**。每放一道指令就要佔用 2 個記憶體位址，因此PC的位址遞增均以 2 為單位。

2.2 Stack 機制（STKPTR、TOS）

- **`STKPTR`** ：記錄目前 Stack 內存放了幾筆資料。
- **`TOS`** ：
  - 執行 `RCALL` 時，會將「下一道指令的位址」推入 TOS。
  - 執行 `RETURN` 時，程式會讀取 TOS 的位址並跳回去繼續執行。

3. 模組化設計：Subroutine 與 Macro

3.1 Subroutine

運作邏輯類似高階語言中的 Function。

運作步驟

1. 定義子程序的標籤（Label）。
2. 使用 `RCALL <Label>` 呼叫並跳轉至子程序執行。
3. 執行完畢時使用 `RETURN` 返回，程式將回到當時 `RCALL` 的下一行繼續執行。

程式範例

```
initial:
    MOVLW   0x03
    MOVWF   0x00
    CLRF    WREG
    rcall   func1       ; 呼叫子程序
    goto    finish

func1:
    ADDWF   0x00, W
    DECFSZ  0x00
    GOTO    func1
    MOVWF   0x01
    RETURN              ; 返回呼叫處的下一行

finish:
    end
```

3.2 Macro（巨集）

用於自訂新的指令模板，在組譯階段展開，支援**帶入參數**。

語法格式

```
macro_name macro p1, p2, p3, p4
    ; macro 內容程式碼
    endm
```

3.3 Macro vs. Subroutine 綜合比較

比較項目	Macro（巨集）	Subroutine（子程序）
參數傳遞	可直接透過語法傳遞參數	呼叫前需手動將參數寫入Register File
編譯/執行行為	Compiler 會在呼叫處**直接複製展開**程式碼	透過跳轉指令**重複呼叫同一段**程式碼
適用情境	適合**簡短**的程式碼	適合**較長、需重複利用**的程式碼

4. 乘法運算 (Multiply)

4.1 Unsigned Multiplication（無號數乘法）

將所有數值視為正數處理，不涉及正負號調整。

MULWF	Multiply W with f
Syntax:	MULWF f {,a}
Operands:	$0 \leq f \leq 255$ $a \in [0,1]$
Operation:	(W) x (f) → PRODH:PRODL
Status Affected:	None
Encoding:	0000 001a ffff ffff
Description:	An unsigned multiplication is carried out between the contents of W and the register file location 'f'. The 16-bit result is stored in the PRODH:PRODL register pair. PRODH contains the high byte. Both W and 'f' are unchanged. None of the Status flags are affected. Note that neither Overflow nor Carry is possible in this operation. A zero result is possible but not detected. If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank (default). If 'a' is '0' and the extended instruction set is enabled, this instruction operates in Indexed Literal Offset Addressing mode whenever

4.2 Signed Multiplication（有號數乘法）

檢查最高有效位元（MSB），若為負數則需依據二補數規則進行符號校正與調整。

公式推導

Signed 8-bit Multiplication

- The multiplication of signed numbers requires the programmer to consider the signs of the multiplier and the multiplicand. Let *M* and *N* represent the magnitudes of two numbers. There are four possible situations:
- Case 1: Both operands are positive (**opl** = *M*, **op2** = *N*). The product of these two operands can be computed by using either the **mulwf f, A** or the **mullw k** instruction.

- Case 2: The first operand is negative (**opl**= -*M*) whereas the second operand is positive (**op2**= *N*). The first operand **opl** will be represented in two's complement of *M* (2^8- *M*) in the PIC18. The product of -*M* and *N*:

$$-M \times N = (2^8 - M) \times N = (2^8 \times N) - M \times N = \underbrace{2^{16} - M \times N}_{①} + \underbrace{2^8 \times N}_{②}$$

- The value 2^{16} is added to this expression. Since in this case the PIC18 is performing a modulo-2^{16} arithmetic (PIC18uses PRODH and PRODL to hold the product), adding the value of 2^{16} makes no difference to the result

$$-M \times N$$

- Case 3: The first operand is positive (**opl**= *M*), but the second operand is negative (**op2**=-*N*). Similar to Case 2, the product of *M* and -*N*:

$$M \times (-N) = M \times (2^8 - N) = (2^8 \times M) - M \times N = \underbrace{2^{16} - M \times N}_{①} + \underbrace{2^8 \times M}_{②}$$

- The first term is the correct product, which is represented as the two's complement of $-M \times N$. The second term of this product is an extra term and can be eliminated by subtracting **opl** from the upper byte of the product of *M* and -*N*.

- Case 4: Both operands are negative (**opl**= -*M*, **op2** = -*N*). The product of -*M* and -*N*:

$$\begin{aligned}(-M) \times (-N) &= (2^8 - M) \times (2^8 - N) = 2^{16} - 2^8 \times M - 2^8 \times N + M \times N \\ &= M \times N + 2^{16} - 2^8 \times M + 2^{16} - 2^8 \times N \\ &= \underbrace{M \times N}_{①} + \underbrace{2^8 \times (2^8 - M)}_{②} + \underbrace{2^8 \times (2^8 - N)}_{③}\end{aligned}$$

- The first term is the product of -*M* and -*N*. The second term and the third term are extra terms and must be eliminated. The second term can be eliminated by subtracting **opl** from the upper byte of the product, whereas the third term can be eliminated by subtracting **op2** from the upper byte of the product.

實際運算步驟

- The signed 8-bitmultiplication can be implemented by the algorithm

Step 1
Multiply two operands (i.e., compute **op1** × **op2**) disregarding their signs.
Step 2
If **op1** is negative, then subtract **op2** from the upper byte of the product.
Step 3
If **op2** is negative, then subtract **op1** from the upper byte of the product.

